Министерство науки и высшего образования Российской Федерации
Пермский филиал федерального государственного бюджетного
образовательного учреждения высшего образования
«Российский экономический университет
имени Г. В. Плеханова»

СОПРОВОДИТЕЛЬНАЯ ЗАПИСКА

к практической работе № 2

на тему

«Разработка калькулятора на C# и WPF»

Выполнил:
студент группы ИПс-21
Ашлапов Иван

Пермь 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Постановка задачи 4
1.1 Средства разработки 4
2 Структура решения 5
3 Создание решения в Visual Studio 6
3.1 Добавление ссылок на проекты 6
3.2 Сборка и запуск приложения 7
4 Разработка библиотеки классов 8
4.1 Класс Calculator 8
4.2 Класс ExpressionCalculator 8
5 Разработка интерфейса WPF 10
5.1 Разметка окна 10
5.2 Модель представления и команды 11
5.3 Ввод с клавиатуры 12
5.4 История вычислений 12
5.5 Продолжение вычислений 13
5.6 Сообщения об ошибках 13
5.7 Проверка элементов интерфейса 13
6 Модульное тестирование 14
6.1 Запуск и результаты тестирования 15
7 Проверка работы программы 16
7.1 Проверка обработки ошибок 17
8 Подготовка проекта к размещению на GitHub 18
ЗАКЛЮЧЕНИЕ 19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 20
ПРИЛОЖЕНИЕ А Класс Calculator 21
ПРИЛОЖЕНИЕ Б Класс ExpressionCalculator 23

ВВЕДЕНИЕ

Практическая работа посвящена созданию калькулятора на языке C# с графическим интерфейсом WPF. Для разделения обязанностей вычислительная часть оформлена как библиотека классов, а её поведение проверяется отдельным проектом модульных тестов. Разработанное решение получило имя AshlapovCalculator.

1 Постановка задачи

Программа должна выполнять сложение, вычитание, умножение, деление и возведение в степень. Выражения вводятся кнопками или с клавиатуры. При вычислении учитываются приоритет операций, вложенные скобки, дробные числа и отрицательные значения.

В таблице 1 представлены основные возможности программы.

Таблица 1 - Основные возможности калькулятора

Возможность	Пример и результат
Приоритет умножения	2+3*4 = 14
Скобки	(2+3)*4 = 20
Степень справа налево	2^3^2 = 512
Минус перед степенью	−2^2 = −4; (−2)^2 = 4
Отрицательная степень	2^−2 = 0,25
Дробные числа	0,1+0,2 = 0,3
Обработка ошибок	5/0 — сообщение о делении на ноль

Для удобства повторных расчётов я предусмотрел историю последних 20 успешных вычислений. После вывода результата пользователь может продолжить расчёт, введя оператор, либо начать новое выражение с цифры. При этом все пять арифметических операций доступны как с помощью кнопок, так и с клавиатуры.

1.1 Средства разработки

C#, .NET 8, WPF, XAML и MSTest 4.0.2. Приложение рассчитано на Windows. Основная логика и модульные тесты имеют целевую платформу net8.0; интерфейс — net8.0-windows.

2 Структура решения

В таблице 2 представлены проекты решения и их назначение.

Таблица 2 - Проекты решения AshlapovCalculator

Проект	Назначение
AshlapovCalculator.Core	Библиотека арифметики и разбора выражений
AshlapovCalculator.Wpf	Окно, стили, модель представления и команды
AshlapovCalculator.Tests	Модульные тесты библиотеки и модели представления

Файлы, которые необходимо создать в каждом проекте, показаны на рисунке 1.

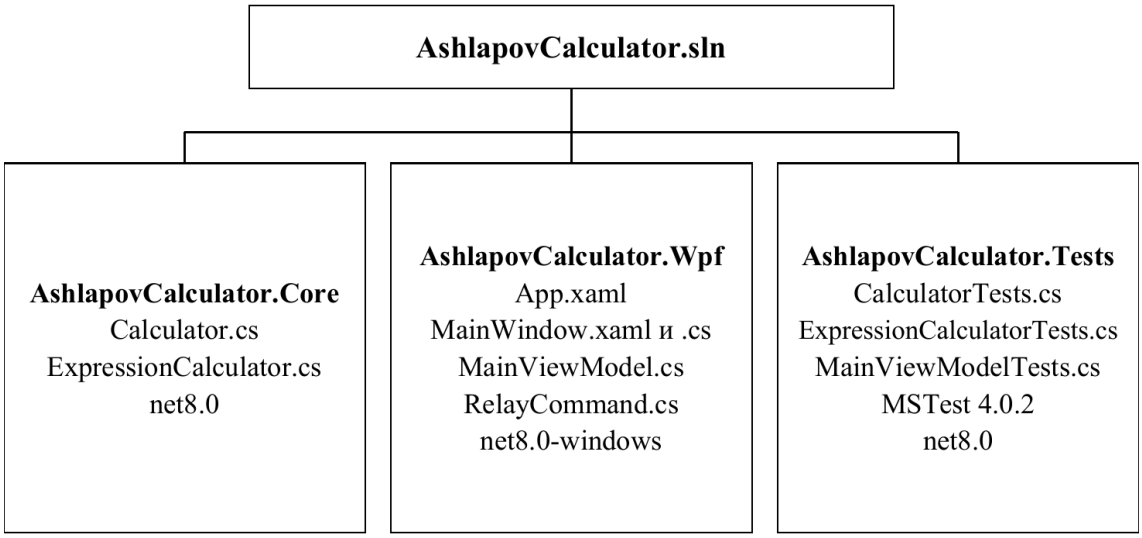


Рисунок 1 - Структура созданного решения AshlapovCalculator

3 Создание решения в Visual Studio

Для создания приложения в Visual Studio выбирается команда «Создать проект». В строке поиска шаблонов вводится WPF, а в фильтре языка выбирается C#. Для данной работы используется шаблон «Приложение WPF» с целевой платформой .NET 8.

В окне настройки задаются имя проекта AshlapovCalculator.Wpf и имя решения AshlapovCalculator. После нажатия «Создать» среда формирует файлы App.xaml и MainWindow.xaml. Первый задаёт запуск приложения и общие ресурсы, второй содержит разметку главного окна.

В обозревателе решений открывается контекстное меню решения и выбирается «Добавить» - «Создать проект». По шаблону «Библиотека классов» создаётся AshlapovCalculator.Core с платформой .NET 8. Заготовка Class1.cs удаляется; через команду добавления класса создаются Calculator.cs и ExpressionCalculator.cs.

Тем же способом в решение добавляется тестовый проект AshlapovCalculator.Tests. В его файле проекта задаётся платформа net8.0 и ссылка на пакет MSTest версии 4.0.2. Создаются файлы CalculatorTests.cs, ExpressionCalculatorTests.cs и MainViewModelTests.cs.

Свойства проектов проверяются перед заполнением классов. Для Core и Tests установлена платформа net8.0. Проект Wpf использует net8.0-windows, параметр UseWPF со значением true и тип вывода WinExe. Такая настройка включает поддержку XAML и запуск графического окна.

3.1 Добавление ссылок на проекты

Листинг 1 - Направления ссылок между проектами

```
AshlapovCalculator.Wpf   -> AshlapovCalculator.Core
AshlapovCalculator.Tests -> AshlapovCalculator.Core
```

В узле «Зависимости» проекта AshlapovCalculator.Wpf выбирается «Добавить ссылку на проект». На вкладке проектов решения отмечается AshlapovCalculator.Core. Это действие повторяется для AshlapovCalculator.Tests. В обоих файлах .csproj появляется элемент ProjectReference с путём к библиотеке Core.

Для проверки модели представления без запуска окна в тестовый проект добавляются MainViewModel.cs и RelayCommand.cs как ссылки на

существующие файлы. В .csproj они перечислены элементами Compile с атрибутами Include и Link. Ссылка на весь проект Wpf тестам не требуется, поскольку эти два класса не используют визуальные элементы WPF.

Связи, полученные после настройки ссылок, показаны на рисунке 2.

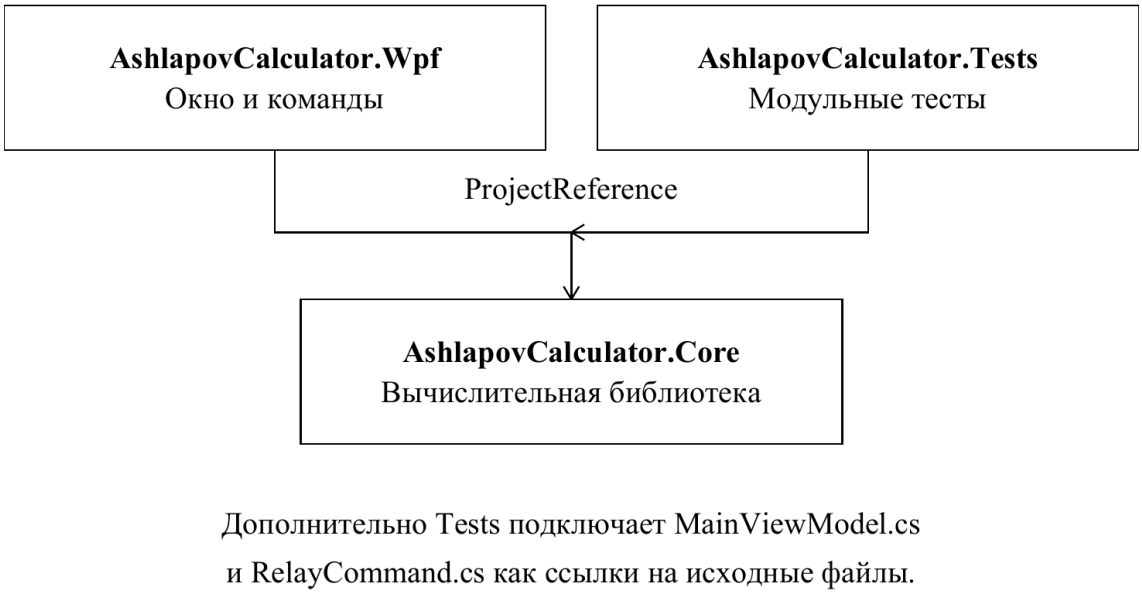


Рисунок 2 - Связи проектов и подключение исходных файлов к тестам

3.2 Сборка и запуск приложения

Для первого запуска открывается AshlapovCalculator.sln и выполняется восстановление пакетов NuGet. В контекстном меню проекта AshlapovCalculator.Wpf выбирается назначение запускаемым проектом. Затем выполняется сборка решения и запуск клавишей F5. При открытии готового репозитория последовательность действий та же.

Из командной строки в папке решения выполняются dotnet restore, затем dotnet build AshlapovCalculator.sln -c Release. Запуск графического приложения: dotnet run --project AshlapovCalculator.Wpf -c Release. Успешная сборка подтверждает, что ссылки между проектами и настройки XAML указаны правильно.

4 Разработка библиотеки классов

4.1 Класс Calculator

В файле Calculator.cs объявляется общедоступный класс Calculator. В него последовательно добавляются методы Add, Subtract, Multiply, Divide и Power, принимающие два аргумента типа double. Для первых четырёх операций используются арифметические операторы C#, для степени - Math.Pow.

Метод Divide перед выполнением операции проверяет делитель. При нулевом значении формируется DivideByZeroException с понятным пользователю сообщением. В методе Calculate(a, operation, b) оператор switch выбирает соответствующий метод; неизвестное обозначение операции приводит к ArgumentException.

Листинг 2 - Фрагмент программы

```
public double Divide(double a, double b)
{
    if (b == 0)
        throw new DivideByZeroException(
            "Деление на ноль невозможно");
    return a / b;
}
```

4.2 Класс ExpressionCalculator

В ExpressionCalculator.cs создаётся класс ExpressionCalculator с методом Calculate(string expression). Сначала проверяется, что строка введена, и удаляются пробелы. Затем SplitExpression последовательно читает символы и формирует списки чисел, унарных знаков и операций. Запятая перед разбором числа заменяется точкой, а числовой формат задаётся через CultureInfo.InvariantCulture.

Для круглых скобок добавляется метод FindClosingBracket. Он подсчитывает глубину вложенности и находит закрывающую скобку для текущей открывающей. Содержимое скобок вычисляется повторным вызовом Calculate, после чего полученное значение участвует в общем выражении как одно число.

Сначала CalculatePowers выполняет степени справа налево. Затем ApplySigns применяет унарные минусы; после этого CalculateOperations обрабатывает умножение и деление, а затем сложение и вычитание. Такой порядок различает −2^2 и (−2)^2.

Листинг 3 - Фрагмент программы

```
2+3*4:  [2, 3, 4], [+, *] -> [2, 12], [+] -> 14
2^3^2:  2^(3^2) = 2^9 = 512
-2^2:   -(2^2) = -4
2^-2:   1/(2^2) = 0,25
```

Ошибки ввода и неверные скобки вызывают исключения. Значения NaN и бесконечность отклоняются. Результат округляется до 10 знаков после запятой, поэтому 0,1+0,2 = 0,3.

Порядок вызова методов для контрольного примера показан на рисунке 3.

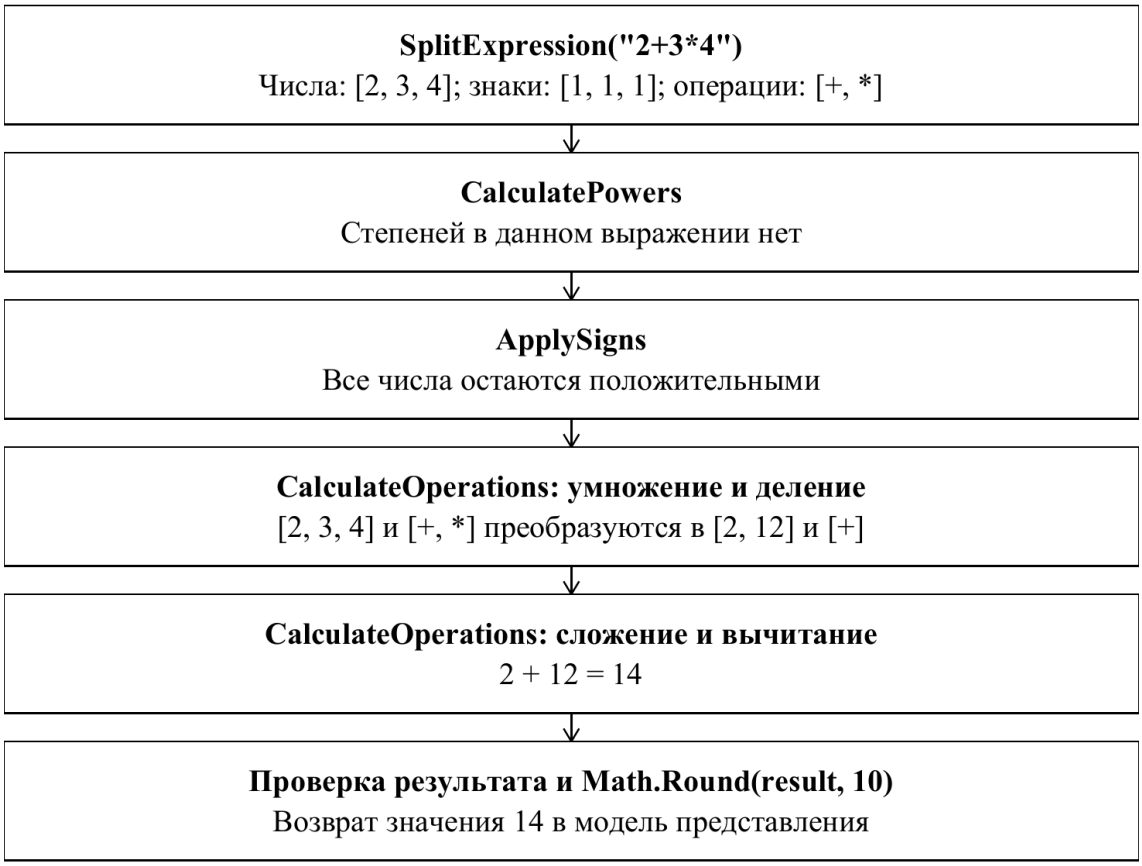


Рисунок 3 - Последовательность вычисления выражения 2+3*4

5 Разработка интерфейса WPF

5.1 Разметка окна

В MainWindow.xaml корневому элементу Window задаются заголовок с фамилией автора, начальный размер 900 × 760 и минимальный размер 760 × 700. Внутри окна создаётся Grid: верхняя строка содержит название и данные автора, центральная область - калькулятор и историю, нижняя строка - подсказку по управлению.

В левой части размещаются дисплей и сетка кнопок из шести строк и четырёх столбцов. Кнопка вычисления занимает всю последнюю строку. Правая часть отведена истории. Цвета, размеры и состояния наведения задаются общими ресурсами и стилями в App.xaml; отдельный стиль применяется к цифрам, операторам, служебным клавишам и кнопке «=».

Для каждой кнопки задаются отображаемый текст, стиль и команда. Цифры, скобки, разделитель и операторы передаются параметром CommandParameter. Кнопки «×» и «÷» передают в вычислитель символы * и /. Кнопки C и удаления последнего символа связываются с отдельными командами очистки и редактирования.

Результат создания окна до выполнения вычислений показан на рисунке 4.

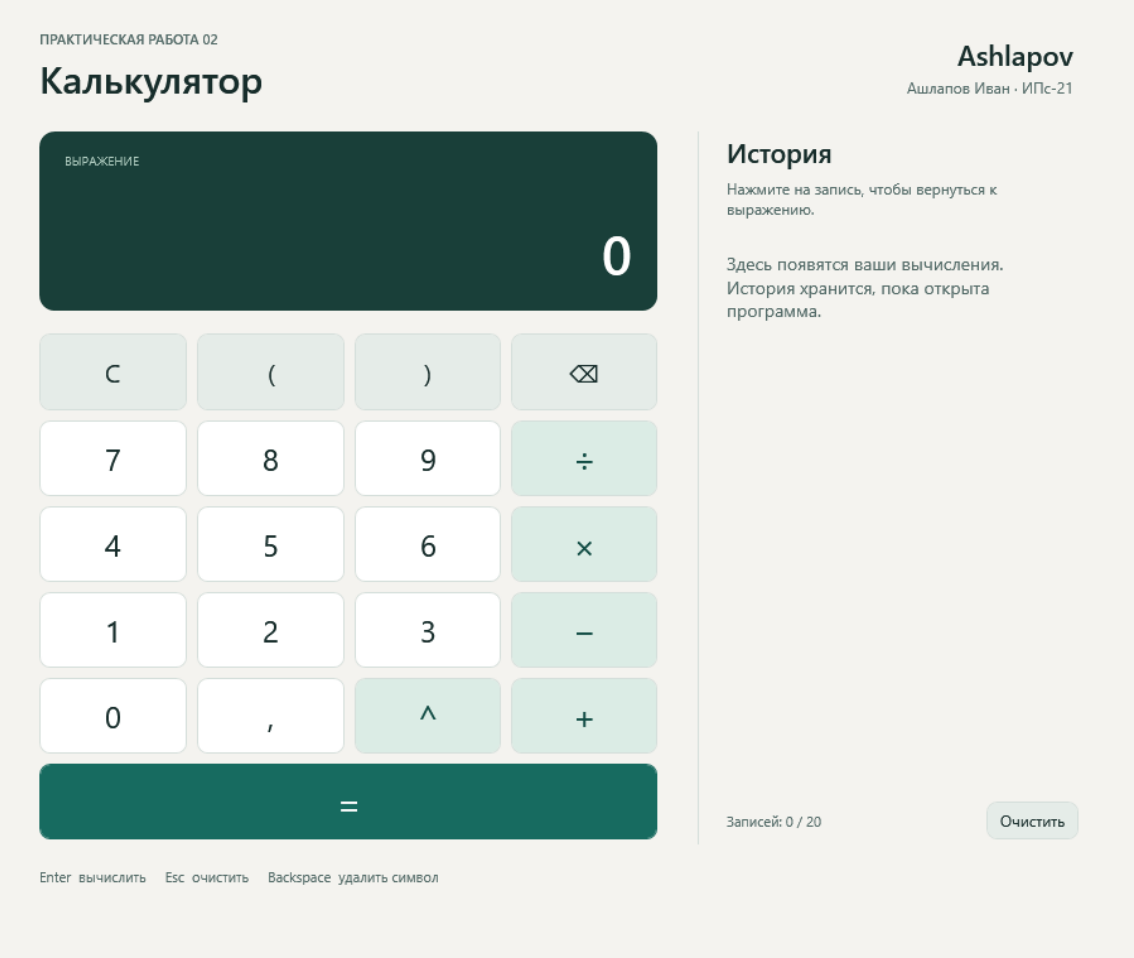


Рисунок 4 - Окно после настройки разметки и привязок

5.2 Модель представления и команды

В проекте Wpf создаются MainViewModel.cs и RelayCommand.cs. Класс MainViewModel реализует INotifyPropertyChanged: при изменении Expression, Result и ErrorMessage вызывается уведомление, после которого WPF обновляет соответствующую привязку на дисплее.

В конструкторе модели создаются команды добавления символа, вычисления, очистки, удаления символа и работы с историей. В Window.DataContext разметки MainWindow.xaml создаётся экземпляр MainViewModel. После этого свойства и команды становятся доступны элементам окна через Binding. Например, кнопка цифры 7 связывается следующим образом.

Листинг 4 - Фрагмент программы

```
<Button Content="7" Style="{StaticResource CalcButton}"
        Command="{Binding AddSymbolCommand}" CommandParameter=
            "7" />
```

RelayCommand реализует ICommand и выполняет переданное действие. Метод Calculate модели вызывает ExpressionCalculator.Calculate, записывает успешный результат и добавляет вычисление в ObservableCollection. При исключении заполняются Result и ErrorMessage. Благодаря уведомлениям и наблюдаемой коллекции дисплей и список истории обновляются сразу.

В MainWindow.xaml.cs подключаются события PreviewTextInput и PreviewKeyDown. Первый обработчик передаёт цифры и знаки в AddSymbol и заменяет точку запятой. Второй связывает Enter с Calculate, Backspace с удалением символа, а Esc с Clear. Обработанное событие помечается Handled, чтобы ввод не выполнялся повторно.

5.3 Ввод с клавиатуры

В таблице 3 представлены клавиши управления и соответствующие действия.

Таблица 3 - Управление с клавиатуры

Клавиша	Действие
0–9, +, −, *, /, ^, (,)	Добавить цифру или оператор
, или .	Добавить десятичный разделитель
Enter или =	Вычислить выражение
Backspace	Удалить последний символ
Esc	Очистить выражение и результат

5.4 История вычислений

После успешного расчёта выражение и результат добавляются в начало истории. В ней сохраняется не более 20 записей; самые старые удаляются автоматически. Повторное нажатие «=» для того же выражения не дублирует последнюю запись. Ошибочные выражения в историю не попадают.

Нажатие на запись возвращает выражение на дисплей для редактирования и повторного расчёта. Клавиша C очищает текущий ввод, сохраняя историю. Отдельная кнопка «Очистить» в колонке истории удаляет только записи. После закрытия приложения история не сохраняется.

5.5 Продолжение вычислений

После 2+3 = 5 нажатие * и затем 4 формирует 5*4 и даёт 20. Если вместо оператора ввести цифру 7, начинается новое выражение 7. Десятичный разделитель после результата начинает новую дробь с нуля.

При продолжении отрицательного результата он заключается в скобки: после 2−4 = −2 ввод ^2 вычисляет (−2)^2 = 4. Для передачи результата в парсер используется запись без экспоненты и с независимым от локали десятичным разделителем.

5.6 Сообщения об ошибках

На дисплее выводится короткое слово «Ошибка», а причина размещается отдельной строкой с переносом текста. Это сохраняет читаемость длинного сообщения. Выражение остаётся доступным для исправления с помощью Backspace либо полной очистки.

5.7 Проверка элементов интерфейса

Проверены начальное состояние, заполненная история, ошибка деления на ноль и минимальный размер окна. Кнопки доступны через Tab и имеют выделение фокуса. При проверке окна ошибки привязок данных WPF не обнаружены.

6 Модульное тестирование

После реализации библиотеки в проекте Tests создаются тестовые классы с атрибутом [TestClass]. Проверки отдельных сценариев отмечаются [TestMethod]. Для нескольких наборов аргументов используются строки [DataRow]. В каждой проверке задаются входные значения, вызывается метод и сравниваются фактический и ожидаемый результаты.

В CalculatorTests проверяются пять операций и исключения, в ExpressionCalculatorTests - разбор строки, скобки, приоритеты и ошибочный ввод. MainViewModelTests проверяет команды, новый ввод после результата, историю и обработку ошибок. Суммарно выполняются 47 проверок. Пример ниже подтверждает отказ от деления на ноль.

В таблице 4 представлены группы модульных тестов.

Таблица 4 - Состав модульных тестов

Класс тестов	Число	Проверяемое поведение
CalculatorTests	8	Пять операций, отрицательная степень, деление на ноль и неизвестная операция
ExpressionCalculatorTests	22	Приоритет, скобки, унарный минус, цепочка степеней, дроби и ошибки выражений
MainViewModelTests	17	Новый ввод, точность продолжения результата, история, очистка и исправление ошибок

Листинг 5 - Фрагмент программы

```
[TestMethod]
public void Divide_ByZero_ThrowsException()
{
    Assert.ThrowsExactly<DivideByZeroException>(
        () => _calculator.Divide(5, 0));
}
```

6.1 Запуск и результаты тестирования

Листинг 6 - Результаты запуска модульных тестов

```
dotnet test AshlapovCalculator.sln -c Release

Всего тестов: 47
Пройдено: 47
Не пройдено: 0
Пропущено: 0
```

Для запуска в Visual Studio открывается окно «Обозреватель тестов» и выбирается «Запустить все тесты». Проверки можно повторить из терминала приведённой выше командой. При ошибке следует открыть сообщение проверки, сопоставить ожидаемое и фактическое значения, исправить реализацию и повторить весь набор.

Проверка выполнена 22.10.2026: пройдены все 47 тестов, пропущенных и неуспешных проверок нет. Дополнительно решение собрано в конфигурации Release без ошибок и предупреждений. Для интерфейса проверены ввод 2+3*4 с результатом 14, очистка через Esc, заполнение истории и вывод ошибки деления на ноль.

7 Проверка работы программы

В истории видны независимые проверки приоритета операций, скобок, цепочки степеней и дробных чисел. Текущее выражение 0,1+0,2 даёт результат 0,3.

Результат проверки представлен на рисунке 5.

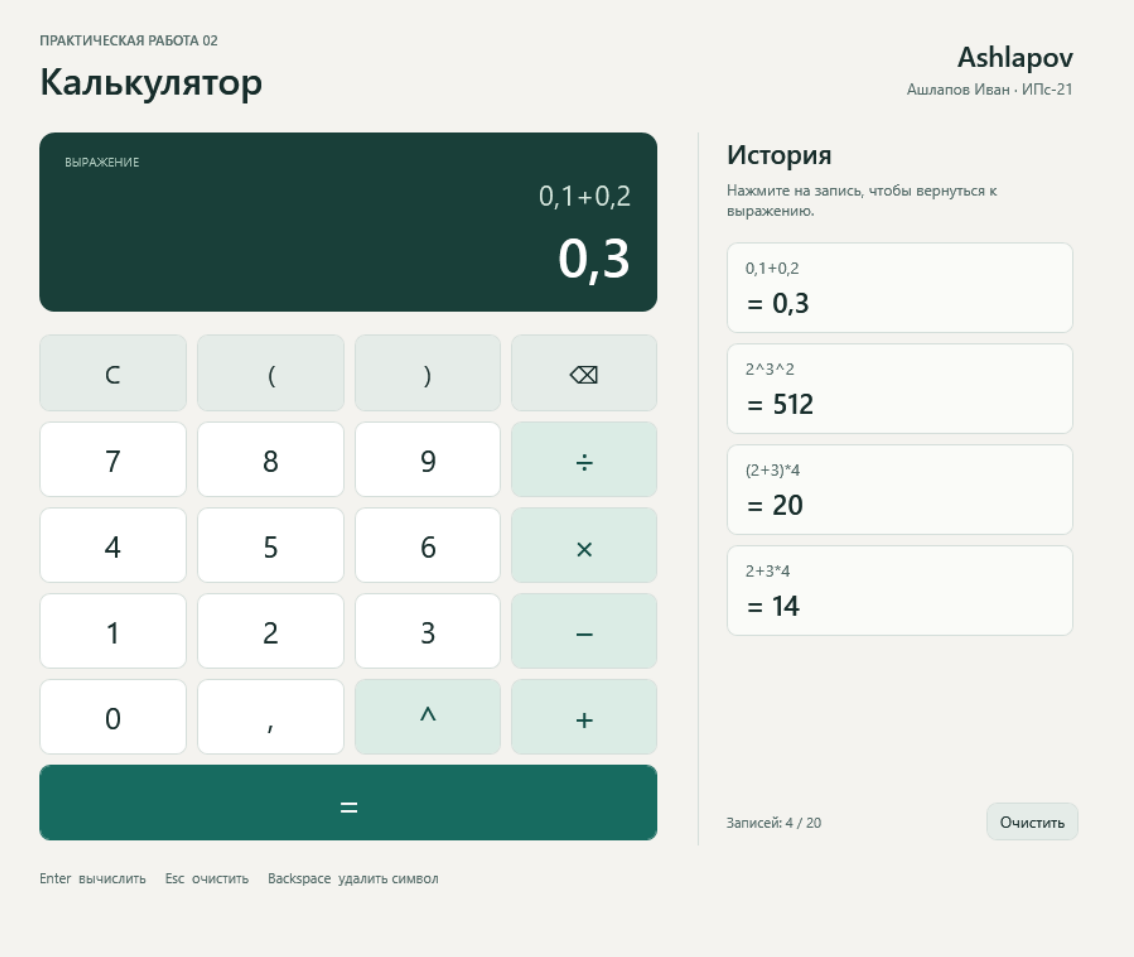


Рисунок 5 - Интерфейс AshlapovCalculator с историей вычислений

Справа новые вычисления располагаются выше предыдущих. Текст автора и группа указаны в заголовке окна и в верхней части интерфейса.

7.1 Проверка обработки ошибок

Попытка вычислить 5/0 приводит к сообщению «Деление на ноль невозможно». История сохраняет предыдущие успешные вычисления, а ошибочная запись в неё не добавляется.

Результат проверки представлен на рисунке 6.

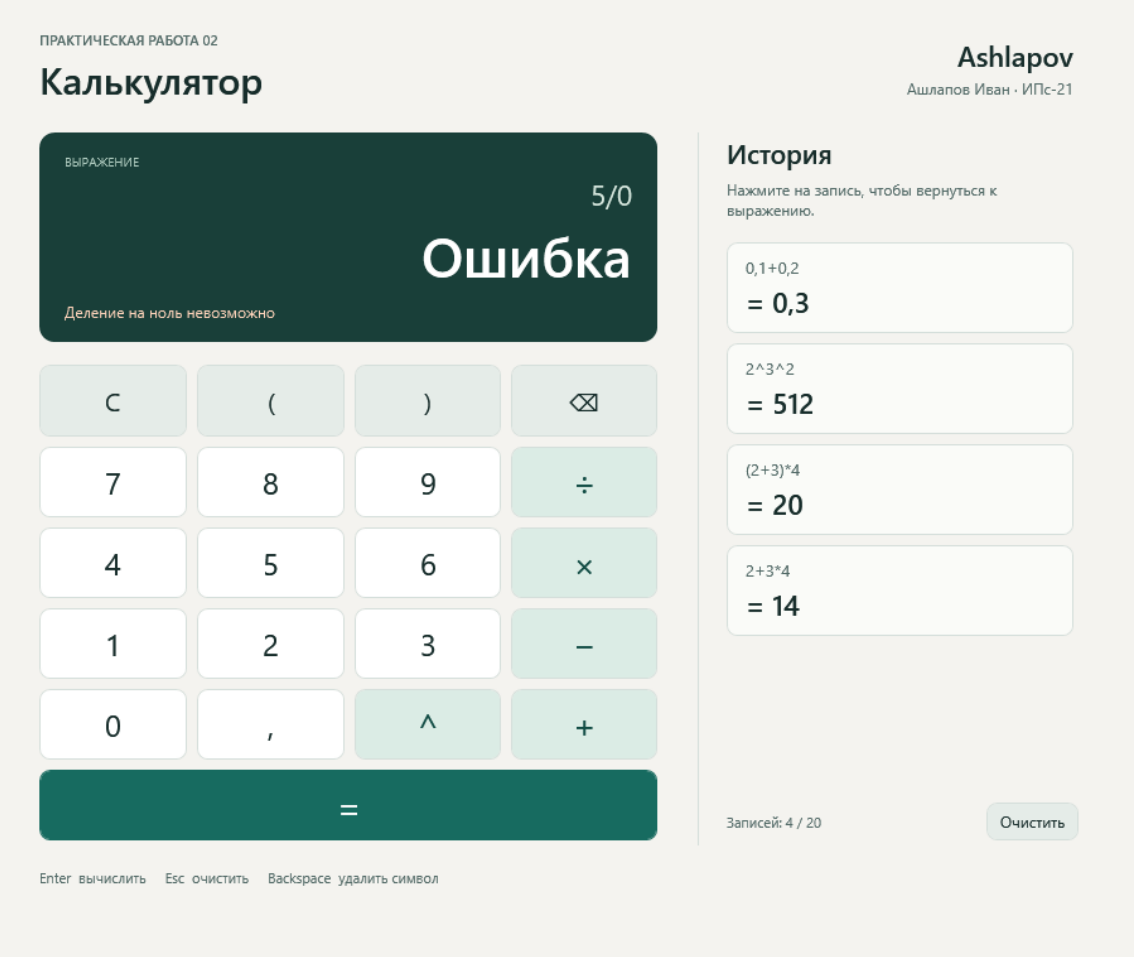


Рисунок 6 - Сообщение об ошибке деления на ноль

Для исправления можно удалить последний символ клавишей Backspace и ввести другой делитель. Например, после замены 0 на 2 выражение 5/2 даёт 2,5. Esc или C полностью очищает текущий ввод.

8 Подготовка проекта к размещению на GitHub

Проект размещён в репозитории https://github.com/aslapovivan989-hue/Ashlapov-Calculator. В корне находятся AshlapovCalculator.sln, README.md и каталоги трёх проектов. Сопроводительная записка в формате PDF и изображение интерфейса находятся в каталоге docs.

Перед публикацией проверяется файл .gitignore: каталоги bin, obj, .vs и TestResults исключаются из отслеживания. После загрузки открываются страница репозитория, инструкция README и PDF-записка. Для повторной проверки достаточно скачать исходные файлы, восстановить зависимости, собрать решение и запустить тесты.

ЗАКЛЮЧЕНИЕ

В результате практической работы я разработал калькулятор с интерфейсом WPF и отдельной библиотекой вычислений. Программа обрабатывает пять арифметических операций, учитывает их приоритет, поддерживает вложенные скобки, дробные и отрицательные числа. Предусмотрены ввод с клавиатуры, история вычислений и продолжение расчёта с полученного результата. Все 47 модульных тестов завершились успешно. Разделение решения на проекты позволяет проверять вычислительную часть независимо от окна приложения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 WPF. Документация и исходный код Microsoft - Текст : электронный. - URL: https://github.com/dotnet/wpf (дата обращения: 22.10.2026).

2 Microsoft Test Framework. Документация MSTest - Текст : электронный. - URL: https://github.com/microsoft/testfx (дата обращения: 22.10.2026).

3 Microsoft Learn. Модульное тестирование C# с помощью MSTest и .NET - Текст : электронный. - URL: https://learn.microsoft.com/dotnet/core/testing/unit-testing-with-mstest (дата обращения: 22.10.2026).

4 Документация GitHub - Текст : электронный. - URL: https://docs.github.com/ru (дата обращения: 22.10.2026).

5 Microsoft Learn. Создание приложения WPF в Visual Studio - Текст : электронный. - URL: https://learn.microsoft.com/dotnet/desktop/wpf/get-started/create-app-visual-studio (дата обращения: 22.10.2026).

ПРИЛОЖЕНИЕ А

(обязательное)

Класс Calculator

AshlapovCalculator.Core/Calculator.cs

```
namespace AshlapovCalculator.Core;

/// <summary>Арифметические операции калькулятора.</summary>
public class Calculator
{
    public double Add(double a, double b)
    {
        return a + b;
    }

    public double Subtract(double a, double b)
    {
        return a - b;
    }

    public double Multiply(double a, double b)
    {
        return a * b;
    }

    public double Divide(double a, double b)
    {
        if (b == 0)
        {
            throw new DivideByZeroException(
                "Деление на ноль невозможно");
        }

        return a / b;
    }

    public double Power(double a, double b)
    {
        return Math.Pow(a, b);
    }

    public double Calculate(double a, char operation, double b)
    {
        switch (operation)
        {
            case '+':
```

```
                return Add(a, b);
            case '-':
                return Subtract(a, b);
            case '*':
                return Multiply(a, b);
            case '/':
                return Divide(a, b);
            case '^':
                return Power(a, b);
            default:
                throw new ArgumentException(
                "Неизвестная операция: " + operation);
        }
    }
}
```

ПРИЛОЖЕНИЕ Б

(обязательное)

Класс ExpressionCalculator

AshlapovCalculator.Core/ExpressionCalculator.cs

```
using System.Globalization;

namespace AshlapovCalculator.Core;

/// <summary>Вычисляет выражение вида "2+3*4" или "(2+3)*4" с
/// учётом приоритета операций и скобок.</summary>
public class ExpressionCalculator
{
    private const string Operations = "+-*/^";

    private readonly Calculator _calculator = new Calculator();

    public double Calculate(string expression)
    {
        if (string.IsNullOrWhiteSpace(expression))
        {
            throw new ArgumentException("Выражение не введено");
        }

        var numbers = new List<double>();
        var signs = new List<int>();
        var operations = new List<char>();
        SplitExpression(expression.Replace(" ", ""), numbers,
            signs, operations);

        // Сначала степень, потом минус перед числом, потом
        // умножение и деление, потом сложение и вычитание.
        CalculatePowers(numbers, signs, operations);
        ApplySigns(numbers, signs);
        CalculateOperations(numbers, operations, "*/");
        CalculateOperations(numbers, operations, "+-");

        double result = numbers[0];
        if (double.IsNaN(result) || double.IsInfinity(result))
        {
            throw new ArithmeticException(
                "Результат невозможно вычислить");
        }

        // Округление убирает погрешность вида 0,1 + 0,2 =
        // 0,30000000000000004.
```

```
        return Math.Round(result, 10);
    }

    // Делит выражение на числа, их знаки и операции: "-2+3*4"
    // -> [2, 3, 4], [-1, 1, 1] и ['+', '*'].
    private void SplitExpression(string expression, List<double>
        numbers, List<int> signs, List<char> operations)
    {
        string number = "";
        int sign = 1;

        for (int i = 0; i < expression.Length; i++)
        {
            char c = expression[i];

            if (char.IsDigit(c) || c == ',' || c == '.')
            {
                number += c;
            }
            else if (c == '(')
            {
                if (number != "")
                {
                    throw new FormatException(
                "Перед скобкой нет знака операции");
                }

                int end = FindClosingBracket(expression, i);
                string inside = expression.Substring(i + 1, end
                - i - 1);
                if (inside == "")
                {
                    throw new FormatException("Пустые скобки");
                }

                // Выражение в скобках вычисляем отдельно и
                // дальше работаем с ним как с обычным числом.
                number =
Calculate(inside).ToString(CultureInfo.InvariantCulture);
                i = end;

                if (i + 1 < expression.Length &&
                !Operations.Contains(expression[i + 1]))
                {
                    throw new FormatException(
                "После скобки нет знака операции");
                }
            }
            else if (c == ')')
            {
```

```
                throw new FormatException(
                "Лишняя закрывающая скобка");
            }
            else if (c == '-' && number == "" && (i == 0 ||
                Operations.Contains(expression[i - 1])))
            {
                // Минус в начале или сразу после знака операции
                // — это минус перед числом: -5, 2*-3.
                sign = -sign;
            }
            else if (Operations.Contains(c))
            {
                numbers.Add(ParseNumber(number));
                signs.Add(sign);
                operations.Add(c);
                number = "";
                sign = 1;
            }
            else
            {
                throw new FormatException(
                "Недопустимый символ: " + c);
            }
        }

        numbers.Add(ParseNumber(number));
        signs.Add(sign);
    }

    private static int FindClosingBracket(string expression, int
        openIndex)
    {
        int depth = 0;

        for (int i = openIndex; i < expression.Length; i++)
        {
            if (expression[i] == '(')
            {
                depth++;
            }
            else if (expression[i] == ')')
            {
                depth--;
                if (depth == 0)
                {
                    return i;
                }
            }
        }

        throw new FormatException(
```

```
            "Не хватает закрывающей скобки");
    }

    private static double ParseNumber(string text)
    {
        text = text.Replace(',', '.');

        if (!double.TryParse(text, NumberStyles.Float,
            CultureInfo.InvariantCulture, out double value))
        {
            throw new FormatException("Некорректное выражение");
        }

        return value;
    }

    // Степень считается справа налево: 2^3^2 = 2^9.
    // Минус перед основанием применяется после степени (-2^2 =
    // -4),
    // а минус перед показателем — сразу (2^-2 = 0,25).
    private void CalculatePowers(List<double> numbers, List<int>
        signs, List<char> operations)
    {
        for (int i = operations.Count - 1; i >= 0; i--)
        {
            if (operations[i] == '^')
            {
                double exponent = signs[i + 1] * numbers[i + 1];
                numbers[i] = _calculator.Power(numbers[i],
                exponent);
                numbers.RemoveAt(i + 1);
                signs.RemoveAt(i + 1);
                operations.RemoveAt(i);
            }
        }
    }

    private static void ApplySigns(List<double> numbers,
        List<int> signs)
    {
        for (int i = 0; i < numbers.Count; i++)
        {
            numbers[i] = signs[i] * numbers[i];
        }
    }

    private void CalculateOperations(List<double> numbers,
        List<char> operations, string symbols)
    {
        int i = 0;
```

```
            while (i < operations.Count)
            {
                if (symbols.Contains(operations[i]))
                {
                    numbers[i] = _calculator.Calculate(numbers[i],
                    operations[i], numbers[i + 1]);
                    numbers.RemoveAt(i + 1);
                    operations.RemoveAt(i);
                }
                else
                {
                    i++;
                }
            }
        }
    }
```